Paradossi della Probabilità

Un Tutorial di Programmazione

Impara Python Simulando l'Impossibile

Liceo Lugano 3 — Giornata Autogestita 2026

In questo tutorial imparerai a **programmare in Python** simulando cinque famosi paradossi della probabilità.

Non serve esperienza di programmazione — solo curiosità!

Alla fine avrai scritto le tue simulazioni,
generato grafici e *dimostrato* che la matematica può sorprendere.

Contents

1 Per Cominciare **3**
- 1.1 Cos'è Python? . . . 3
- 1.2 Configurazione . . . 3
- 1.3 Il Tuo Primo Programma . . . 3
- 1.4 Fondamenti di Python (5 minuti) . . . 3
- 1.5 Importare Librerie . . . 4

2 Paradosso I: Il Problema di Monty Hall **5**
- 2.1 Passo 1: Simulare Una Partita . . . 5
- 2.2 Passo 2: Simulare 1000 Partite . . . 6
- 2.3 Passo 3: Osservare la Convergenza . . . 6
- 2.4 Passo 4: Grafico a Barre . . . 7

3 Paradosso II: Il Problema del Ragazzo o della Ragazza **9**
- 3.1 Perché 1/3? Ragioniamoci . . . 9
- 3.2 Passo 1: Simulare Famiglie Casuali . . . 9
- 3.3 Passo 2: Filtrare e Contare . . . 10
- 3.4 Passo 3: Grafico di Convergenza . . . 10

4 Paradosso III: Il Problema del Compleanno **12**
- 4.1 Passo 1: Simulare Una Stanza . . . 12
- 4.2 Passo 2: Stimare la Probabilità . . . 12
- 4.3 Passo 3: La Formula Esatta . . . 13
- 4.4 Passo 4: Il Grafico . . . 13

5 Paradosso IV: Il Paradosso di Simpson **15**
5.1 Il Trucco: Tassi di Base Diversi . . . 15
5.2 Passo 1: Creare i Dati . . . 15
5.3 Passo 2: Analizzare i Dati . . . 16
5.4 Passo 3: Visualizzare . . . 16

6 Paradosso V: Sei Gradi di Separazione **18**
6.1 Passo 1: La Potenza della Crescita Esponenziale . . . 18
6.2 Passo 2: Costruire una Rete Sociale . . . 18
6.3 Passo 3: Trovare il Cammino Più Breve (BFS) . . . 19
6.4 Passo 4: Misurare Tutte le Distanze . . . 20
6.5 Passo 5: Istogramma delle Distanze . . . 20

7 Riepilogo Finale **22**

8 Cosa Hai Imparato **24**
8.1 Punti Chiave . . . 24
8.2 Per Approfondire . . . 24

Per Cominciare

Cos'è Python?

Python è uno dei linguaggi di programmazione più diffusi al mondo. Scienziati, ingegneri, analisti di dati e persino artisti lo usano ogni giorno. Perché?

- Si legge quasi come l'inglese
- È gratuito e funziona su qualsiasi computer
- Ha strumenti potenti per matematica, grafici e simulazioni

Configurazione

Il modo più semplice per iniziare è **Google Colab** — non serve installare nulla!

1. Vai su https://colab.research.google.com
2. Accedi con il tuo account Google
3. Clicca su **"Nuovo notebook"**
4. Sei pronto a programmare!

Importante

In Colab si scrive il codice in *celle*. Premi **Shift** + **Enter** per eseguire una cella. Provalo subito!

Il Tuo Primo Programma

Scrivi questo in una cella e premi **Shift** + **Enter**:

```python
print("Hello, world!")
print("I am learning Python!")
print(2 + 3)
```

Output

```
Hello, world!
I am learning Python!
5
```

Complimenti — hai appena scritto il tuo primo programma!

Fondamenti di Python (5 minuti)

Ecco tutto ciò che ti serve per questo tutorial:

```python
# Variabili --- memorizzano valori con un nome
name = "Alice"
age = 17
pi = 3.14159

# Print --- mostra i risultati a schermo
```

```python
print(name)                # Alice
print(f"I am {age}")       # I am 17

# Matematica --- Python è una calcolatrice
print(10 / 3)              # 3.3333...
print(2 ** 10)             # 1024 (potenze!)

# Liste --- collezioni ordinate
colors = ["red", "blue", "green"]
print(colors[0])           # red (si conta da 0!)
print(len(colors))         # 3

# Cicli --- ripetere azioni
for i in range(5):
    print(i)               # stampa 0, 1, 2, 3, 4

# Condizioni --- prendere decisioni
x = 7
if x > 5:
    print("big")
else:
    print("small")
```

L'indentazione conta!

In Python, gli spazi all'inizio di una riga *non sono decorazione* — definiscono la struttura. Tutto ciò che sta dentro un ciclo `for` o un `if` deve essere indentato (4 spazi). Colab lo fa automaticamente per te.

Importare Librerie

Python ha migliaia di *librerie* — strumenti già pronti che puoi usare. Ne servono due:

```python
import random               # per generare numeri casuali
import matplotlib.pyplot as plt  # per creare grafici
```

In Colab queste sono già installate. Basta eseguire questa cella una volta all'inizio del notebook.

La libreria `random`

`random` permette di simulare il caso:

- `random.random()` — un numero casuale tra 0 e 1
- `random.randint(1, 6)` — un intero casuale (come lanciare un dado)
- `random.choice(["a", "b", "c"])` — sceglie un elemento a caso da una lista

Prova:

```python
import random

# Lancia un dado 10 volte
for i in range(10):
    roll = random.randint(1, 6)
    print(f"Roll {i+1}: {roll}")
```

Eseguilo più volte — otterrai risultati diversi ogni volta! Questa è la casualità.

Paradosso I: Il Problema di Monty Hall

La Storia

Sei in un quiz televisivo. Ci sono **3 porte**. Dietro una c'è un'**auto**, dietro le altre due ci sono delle **capre**. Scegli una porta. Il conduttore (che *sa* cosa c'è dietro ogni porta) apre un'altra porta, rivelando una capra. Poi ti chiede:

"Vuoi cambiare porta?"

La maggior parte delle persone dice che non fa differenza — è 50/50, no?
Sbagliato. Cambiare vince **2/3** delle volte!

Sembra impossibile. *Dimostriamolo* con il codice.

Passo 1: Simulare Una Partita

Pensiamo a cosa succede in una partita:

1. L'auto viene piazzata dietro una porta a caso (0, 1 o 2)
2. Il giocatore sceglie una porta a caso
3. Il conduttore apre una porta con una capra (non quella del giocatore, non quella dell'auto)
4. Il giocatore decide: **restare** o **cambiare**

```python
import random

# Piazza l'auto dietro una porta casuale
car = random.randint(0, 2)
print(f"Car is behind door {car}")

# Il giocatore sceglie una porta
player_pick = random.randint(0, 2)
print(f"Player picks door {player_pick}")

# Il giocatore vince RESTANDO?
if player_pick == car:
    print("STAY wins! (player picked the car)")
else:
    print("SWITCH wins! (player didn't pick the car)")
```

L'Intuizione Chiave

Nota una cosa: vinci **restando** solo se la tua prima scelta era l'auto. Vinci **cambiando** ogni volta che la tua prima scelta *non* era l'auto. Dato che ci sono 3 porte:

- Probabilità di indovinare l'auto al primo colpo: **1/3**
- Probabilità di *non* indovinare l'auto: **2/3**

Quindi cambiare vince 2/3 delle volte!

Passo 2: Simulare 1000 Partite

Una sola partita non dimostra molto. Giochiamo **1000 partite** e contiamo:

```python
import random

stay_wins = 0
switch_wins = 0

for game in range(1000):
    car = random.randint(0, 2)          # piazza l'auto
    player_pick = random.randint(0, 2)  # il giocatore sceglie

    if player_pick == car:
        stay_wins += 1      # restare avrebbe vinto
    else:
        switch_wins += 1    # cambiare avrebbe vinto

print(f"Games played: 1000")
print(f"Stay wins:   {stay_wins}  ({stay_wins/10:.1f}%)")
print(f"Switch wins: {switch_wins} ({switch_wins/10:.1f}%)")
```

Output

```
Games played:  1000
Stay wins:  332 (33.2%)
Switch wins:  668 (66.8%)
```

I numeri parlano da soli: cambiare vince circa il **67%** delle volte!

Passo 3: Osservare la Convergenza

Creiamo un bel grafico che mostra come le percentuali di vittoria *convergono* a $1/3$ e $2/3$ man mano che giochiamo più partite:

```python
import random
import matplotlib.pyplot as plt

n_games = 5000
stay_wins = 0
switch_wins = 0
stay_rates = []
switch_rates = []

for game in range(1, n_games + 1):
    car = random.randint(0, 2)
    pick = random.randint(0, 2)

    if pick == car:
        stay_wins += 1
    else:
        switch_wins += 1

    # Registra la percentuale di vittoria corrente
    stay_rates.append(stay_wins / game)
    switch_rates.append(switch_wins / game)

# Grafico!
plt.figure(figsize=(10, 5))
```

```
plt.plot(stay_rates, color='#DC2626', linewidth=1, label='Stay', alpha=0.8)
plt.plot(switch_rates, color='#16A34A', linewidth=1, label='Switch', alpha=0.8)
plt.axhline(y=1/3, color='#DC2626', linestyle='--', alpha=0.5, label='Theory: 1/3')
plt.axhline(y=2/3, color='#16A34A', linestyle='--', alpha=0.5, label='Theory: 2/3')
plt.xlabel('Number of games', fontsize=12)
plt.ylabel('Win rate', fontsize=12)
plt.title('Monty Hall: Stay vs Switch', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.ylim(0.15, 0.85)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Dovresti vedere la linea rossa (restare) stabilizzarsi intorno al 33% e la linea verde (cambiare) intorno al 67%. Più partite giochi, più i valori si avvicinano a quelli teorici.

Cosa hai appena fatto

Hai usato una **simulazione Monte Carlo**: esperimenti casuali ripetuti per stimare le probabilità. È una tecnica reale usata ogni giorno da scienziati, ingegneri e analisti finanziari!

Passo 4: Grafico a Barre

Creiamo un grafico a barre per riassumere i risultati finali:

```
strategies = ['Stay', 'Switch']
results = [stay_wins / n_games, switch_wins / n_games]
theory = [1/3, 2/3]
colors = ['#DC2626', '#16A34A']

fig, ax = plt.subplots(figsize=(6, 4))
x = [0, 1]
bars = ax.bar([i - 0.15 for i in x], results, 0.3,
              color=colors, alpha=0.8, label='Simulation')
ax.bar([i + 0.15 for i in x], theory, 0.3,
       color=colors, alpha=0.3, label='Theory')

for i, (r, t) in enumerate(zip(results, theory)):
    ax.text(i - 0.15, r + 0.02, f'{r:.1%}', ha='center',
            fontweight='bold', fontsize=11)
    ax.text(i + 0.15, t + 0.02, f'{t:.1%}', ha='center',
            fontsize=10, color='gray')

ax.set_xticks(x)
ax.set_xticklabels(strategies, fontsize=12)
ax.set_ylabel('Win Rate', fontsize=12)
ax.set_title('Monty Hall: Final Results', fontsize=14, fontweight='bold')
ax.set_ylim(0, 0.85)
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

Sfida: 100 Porte!

E se ci fossero **100 porte** invece di 3? Ne scegli una, il conduttore apre 98 porte con capre, lasciando la tua porta e un'altra. Cambieresti?
Modifica la simulazione:

```
n_doors = 100
car = random.randint(0, n_doors - 1)
pick = random.randint(0, n_doors - 1)
# Restare vince solo se hai indovinato: 1/100
# Cambiare vince le altre 99/100 volte!
```

Prova con 10, 50 e 100 porte. Che schema noti?

Paradosso II: Il Problema del Ragazzo o della Ragazza

La Storia

Un genitore ti dice: *"Ho due figli. Almeno uno di loro è un maschio."*

Qual è la probabilità che *entrambi* siano maschi?

La maggior parte delle persone risponde 1/2. Dopotutto, l'altro figlio è o maschio o femmina, no?
La risposta è 1/3.

Perché 1/3? Ragioniamoci

Una famiglia con due figli ha quattro possibilità equiprobabili:

Caso	Figlio 1	Figlio 2	Almeno un maschio?
1	Femmina	Femmina	No
2	Femmina	Maschio	Sì
3	Maschio	Femmina	Sì
4	Maschio	Maschio	Sì

Sappiamo che "almeno uno è maschio", quindi il Caso 1 è eliminato. Dei 3 casi rimanenti, solo 1 ha entrambi maschi. Dunque la probabilità è $\frac{1}{3}$!

Passo 1: Simulare Famiglie Casuali

```python
import random

def make_family():
    """Create a random family with 2 children."""
    child1 = random.choice(["Boy", "Girl"])
    child2 = random.choice(["Boy", "Girl"])
    return (child1, child2)

# Genera 10 famiglie e mostrale
for i in range(10):
    family = make_family()
    print(f"Family {i+1}: {family[0]}, {family[1]}")
```

Output

```
Family 1:  Boy, Girl
Family 2:  Girl, Girl
Family 3:  Boy, Boy
Family 4:  Girl, Boy
Family 5:  Boy, Girl
...
```

Passo 2: Filtrare e Contare

Ora generiamo molte famiglie, teniamo solo quelle con almeno un maschio e contiamo quante hanno *entrambi* maschi:

```python
import random

n_families = 10000
at_least_one_boy = 0
both_boys = 0

for i in range(n_families):
    child1 = random.choice(["Boy", "Girl"])
    child2 = random.choice(["Boy", "Girl"])

    # Questa famiglia ha almeno un maschio?
    if child1 == "Boy" or child2 == "Boy":
        at_least_one_boy += 1

        # Sono ENTRAMBI maschi?
        if child1 == "Boy" and child2 == "Boy":
            both_boys += 1

probability = both_boys / at_least_one_boy
print(f"Families with at least one boy: {at_least_one_boy}")
print(f"Of those, both boys: {both_boys}")
print(f"Probability: {probability:.4f}")
print(f"Theory: {1/3:.4f}")
```

Output

```
Families with at least one boy:  7509
Of those, both boys:  2498
Probability:  0.3327
Theory:  0.3333
```

Passo 3: Grafico di Convergenza

```python
import random
import matplotlib.pyplot as plt

n = 10000
at_least_one_boy = 0
both_boys = 0
probabilities = []

for i in range(n):
    c1 = random.choice(["Boy", "Girl"])
    c2 = random.choice(["Boy", "Girl"])

    if c1 == "Boy" or c2 == "Boy":
        at_least_one_boy += 1
        if c1 == "Boy" and c2 == "Boy":
            both_boys += 1

    # Registra la probabilità corrente (evita divisione per zero)
    if at_least_one_boy > 0:
        probabilities.append(both_boys / at_least_one_boy)
```

```
    else:
        probabilities.append(0)

plt.figure(figsize=(10, 5))
plt.plot(probabilities, color='#2563EB', linewidth=1, alpha=0.8,
         label=f'Simulation (final: {probabilities[-1]:.4f})')
plt.axhline(y=1/3, color='#DC2626', linestyle='--', linewidth=2,
            label='Theory: 1/3')
plt.xlabel('Number of families', fontsize=12)
plt.ylabel('P(both boys | at least one boy)', fontsize=12)
plt.title('Boy or Girl Paradox', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.ylim(0.2, 0.5)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Sfida: Una Domanda Diversa

E se il genitore dicesse: "*Il mio figlio **più grande** è un maschio*"? Ora qual è la probabilità che entrambi siano maschi?
Suggerimento: Stavolta fissiamo Figlio 1 = Maschio. L'unica domanda riguarda il Figlio 2. Scrivi la simulazione e osserva — la risposta diventa $\mathbf{1/2}$!
Perché specificare *quale* figlio cambia la risposta?

Paradosso III: Il Problema del Compleanno

La Storia

Quante persone servono in una stanza perché ci sia una **probabilità superiore al 50%** che due di loro condividano il compleanno?
La maggior parte delle persone dice circa 180 (la metà di 365). La vera risposta?

Solo 23 persone!

Con sole 23 persone, c'è il 50,7% di probabilità che almeno due condividano il compleanno. Con 50 persone, si arriva al 97%!

Passo 1: Simulare Una Stanza

Generiamo compleanni casuali per un gruppo e verifichiamo se ci sono coincidenze:

```python
import random

def has_birthday_match(n_people):
    """Check if any two people in a group share a birthday."""
    birthdays = []
    for i in range(n_people):
        birthday = random.randint(1, 365)  # giorno dell'anno
        if birthday in birthdays:
            return True  # coincidenza trovata!
        birthdays.append(birthday)
    return False  # nessuna coincidenza

# Proviamo con 23 persone
result = has_birthday_match(23)
print(f"Match found: {result}")
```

Esegui questa cella più volte. A volte otterrai `True`, a volte `False`. Questo perché 23 persone danno circa il lancio di una moneta (50%).

Passo 2: Stimare la Probabilità

Per stimare la probabilità, ripetiamo l'esperimento molte volte:

```python
import random

def birthday_probability(n_people, n_trials=10000):
    """Estimate P(match) for a group of n_people."""
    matches = 0
    for trial in range(n_trials):
        if has_birthday_match(n_people):
            matches += 1
    return matches / n_trials

# Testiamo con diverse dimensioni del gruppo
for n in [10, 20, 23, 30, 40, 50, 60]:
    p = birthday_probability(n)
    print(f"{n:3d} people: P(match) = {p:.1%}")
```

Output

```
10 people:  P(match) = 11.8%
20 people:  P(match) = 41.0%
23 people:  P(match) = 50.4%
30 people:  P(match) = 70.5%
40 people:  P(match) = 89.2%
50 people:  P(match) = 97.1%
60 people:  P(match) = 99.4%
```

Passo 3: La Formula Esatta

Possiamo anche calcolare la probabilità *esatta* matematicamente:

$$P(\text{nessuna coincidenza con } n \text{ persone}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \cdots \times \frac{365-n+1}{365}$$

$$P(\text{almeno una coincidenza}) = 1 - P(\text{nessuna coincidenza})$$

In Python:

```python
def birthday_theory(n):
    """Exact P(at least one shared birthday) for n people."""
    p_no_match = 1.0
    for k in range(1, n):
        p_no_match *= (365 - k) / 365
    return 1 - p_no_match

print(f"P(match with 23 people) = {birthday_theory(23):.4f}")
```

Output

```
P(match with 23 people) = 0.5073
```

Passo 4: Il Grafico

Ora creiamo la classica curva del Problema del Compleanno, confrontando simulazione e teoria:

```python
import random
import matplotlib.pyplot as plt

# Simulazione
group_sizes = list(range(2, 61))
simulated = [birthday_probability(n, 5000) for n in group_sizes]

# Teoria
theoretical = [birthday_theory(n) for n in group_sizes]

# Grafico
plt.figure(figsize=(10, 5))
plt.scatter(group_sizes, simulated, s=20, color='#2563EB',
            alpha=0.6, zorder=3, label='Simulation (5000 trials)')
plt.plot(group_sizes, theoretical, color='#DC2626',
         linewidth=2, zorder=2, label='Exact theory')
plt.axhline(y=0.5, color='gray', linestyle=':', linewidth=1)
plt.axvline(x=23, color='#16A34A', linestyle='--', linewidth=1.5,
            alpha=0.8, label='n = 23 (50.7%)')
```

```
plt.xlabel('Number of people in the room', fontsize=12)
plt.ylabel('P(at least one shared birthday)', fontsize=12)
plt.title('The Birthday Problem', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Perché è così sorprendente?

La nostra intuizione fallisce perché pensiamo alla possibilità che il *nostro* compleanno coincida con quello di qualcuno. Ma la domanda è se *due qualsiasi* persone coincidano. Con 23 persone ci sono $\binom{23}{2} = 253$ coppie possibili da confrontare — sono tantissime possibilità di coincidenza!

Sfida: La Tua Classe!

1. Quanti studenti ci sono nella tua classe? Usa la formula per calcolare la probabilità teorica di una coincidenza di compleanno.
2. Raccogli i compleanni reali dei tuoi compagni. C'è una coincidenza? Confronta con la teoria!
3. E se includessimo anche l'anno? Cambierebbe qualcosa?

Paradosso IV: Il Paradosso di Simpson

La Storia

Nel 1973, l'Università della California a Berkeley fu accusata di **discriminazione di genere** nelle ammissioni ai corsi di dottorato. I numeri complessivi sembravano chiari:

	Uomini	**Donne**
Candidature	8.442	4.321
Ammessi	**44%**	**35%**

Caso chiuso? Non così in fretta! Quando i ricercatori analizzarono *ogni dipartimento separatamente*, scoprirono che le donne avevano tassi di ammissione **più alti** nella maggior parte dei dipartimenti!
Com'è possibile che le donne vadano meglio *ovunque* ma peggio *nel complesso*?

Il Trucco: Tassi di Base Diversi

Il segreto: le donne facevano domanda soprattutto ai dipartimenti *competitivi* (con bassi tassi di ammissione), mentre gli uomini facevano domanda soprattutto ai dipartimenti *facili*. Simuliamolo.

Passo 1: Creare i Dati

```python
import random

def simulate_admissions(n_applicants=10000):
    """Simulate Berkeley-like admissions data."""
    results = {"men": {"easy": [], "hard": []},
               "women": {"easy": [], "hard": []}}

    for i in range(n_applicants):
        # Metà uomini, metà donne
        gender = "men" if i < n_applicants // 2 else "women"

        # Gli uomini fanno domanda soprattutto al dipartimento facile (80%)
        # Le donne fanno domanda soprattutto al dipartimento difficile (80%)
        if gender == "men":
            dept = "easy" if random.random() < 0.80 else "hard"
        else:
            dept = "easy" if random.random() < 0.20 else "hard"

        # Tassi di ammissione: dip. facile ~60%, dip. difficile ~20%
        # Le donne hanno un LEGGERO VANTAGGIO in entrambi (+2%)
        if dept == "easy":
            rate = 0.62 if gender == "women" else 0.60
        else:
            rate = 0.22 if gender == "women" else 0.20

        admitted = random.random() < rate
        results[gender][dept].append(admitted)

```

```
    return results

data = simulate_admissions()
```

Passo 2: Analizzare i Dati

```
def admission_rate(lst):
    """Calculate admission rate from a list of True/False."""
    if len(lst) == 0:
        return 0
    return sum(lst) / len(lst)

# Tassi per dipartimento
print("=== DEPARTMENT LEVEL ===")
for dept in ["easy", "hard"]:
    m = admission_rate(data["men"][dept])
    w = admission_rate(data["women"][dept])
    winner = "Women win!" if w > m else "Men win!"
    print(f"{dept.upper():5s} dept: Men={m:.1%}, Women={w:.1%}  --> {winner}")

# Tassi complessivi
print("\n=== OVERALL ===")
all_men = data["men"]["easy"] + data["men"]["hard"]
all_women = data["women"]["easy"] + data["women"]["hard"]
m_all = admission_rate(all_men)
w_all = admission_rate(all_women)
print(f"Overall:    Men={m_all:.1%}, Women={w_all:.1%}")
print(f"Men applied to easy dept: {len(data['men']['easy'])}")
print(f"Women applied to easy dept: {len(data['women']['easy'])}")
```

Output

```
=== DEPARTMENT LEVEL ===
EASY dept:  Men=60.2%, Women=62.4% -> Women win!
HARD dept:  Men=20.1%, Women=22.3% -> Women win!

=== OVERALL ===
Overall:  Men=52.1%, Women=30.2%
Men applied to easy dept:  4010
Women applied to easy dept:  1005
```

Le donne vincono in *ogni* dipartimento, ma perdono nel complesso! Ecco il Paradosso di Simpson.

Passo 3: Visualizzare

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(12, 4.5), sharey=True)

categories = ["Men", "Women"]
colors = ['#2563EB', '#DC2626']

# Dip. facile
rates = [admission_rate(data["men"]["easy"]),
         admission_rate(data["women"]["easy"])]
```

```python
axes[0].bar(categories, rates, color=colors, alpha=0.8)
axes[0].set_title("Easy Department", fontweight='bold')
axes[0].set_ylabel("Admission Rate")
for i, r in enumerate(rates):
    axes[0].text(i, r + 0.01, f'{r:.1%}', ha='center', fontweight='bold')

# Dip. difficile
rates = [admission_rate(data["men"]["hard"]),
         admission_rate(data["women"]["hard"])]
axes[1].bar(categories, rates, color=colors, alpha=0.8)
axes[1].set_title("Hard Department", fontweight='bold')
for i, r in enumerate(rates):
    axes[1].text(i, r + 0.01, f'{r:.1%}', ha='center', fontweight='bold')

# Complessivo
rates = [admission_rate(all_men), admission_rate(all_women)]
axes[2].bar(categories, rates, color=colors, alpha=0.8)
axes[2].set_title("Overall", fontweight='bold')
for i, r in enumerate(rates):
    axes[2].text(i, r + 0.01, f'{r:.1%}', ha='center', fontweight='bold')

# Annotazioni
axes[0].text(0.5, 0.95, 'Women higher!', transform=axes[0].transAxes,
             ha='center', color='green', fontweight='bold', fontsize=11)
axes[1].text(0.5, 0.95, 'Women higher!', transform=axes[1].transAxes,
             ha='center', color='green', fontweight='bold', fontsize=11)
axes[2].text(0.5, 0.95, 'Men higher!?', transform=axes[2].transAxes,
             ha='center', color='red', fontweight='bold', fontsize=11)

for ax in axes:
    ax.set_ylim(0, 0.85)
    ax.grid(axis='y', alpha=0.3)

plt.suptitle("Simpson's Paradox", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

La Lezione

I dati aggregati possono essere fuorvianti. I numeri complessivi nascondono il fatto che le donne sceglievano dipartimenti più difficili. La "variabile confondente" (la scelta del dipartimento) inverte la tendenza. Ecco perché gli scienziati cercano sempre variabili nascoste prima di trarre conclusioni!
Succede ovunque: medicina, sport, economia, politica...

Sfida: Crea il Tuo Paradosso

Riesci a creare uno scenario con **tre** dipartimenti in cui il paradosso si verifica ancora? Che ne dici di un esempio sportivo: un giocatore con una media battuta migliore in ogni stagione ma una media complessiva peggiore?

Paradosso V: Sei Gradi di Separazione

La Storia

L'idea è semplice e sorprendente: **due persone qualsiasi sulla Terra** possono essere collegate attraverso al massimo circa **6 conoscenze intermedie**.
Tu conosci qualcuno, che conosce qualcuno, che conosce qualcuno... e in soli 6 passaggi puoi raggiungere chiunque — il Presidente degli Stati Uniti, un contadino in Mongolia, un pescatore in Brasile.
Facebook lo ha confermato nel 2016: la distanza media tra due utenti qualsiasi era di **3,57 passaggi**.

Passo 1: La Potenza della Crescita Esponenziale

Se ogni persona conosce circa 150 persone (il numero di Dunbar), quante persone puoi raggiungere?

```python
contacts = 150

print("Steps | People reachable")
print("-" * 30)
for step in range(7):
    reach = contacts ** step
    print(f"  {step}   | {reach:>15,}")

print(f"\nWorld population: 8,000,000,000")
print(f"Reached in 5 steps: {contacts**5:,}")
```

Output

```
Steps | People reachable
---------------
0 | 1
1 | 150
2 | 22,500
3 | 3,375,000
4 | 506,250,000
5 | 75,937,500,000

World population:  8,000,000,000
Reached in 5 steps:  75,937,500,000
```

Al passo 5, puoi *teoricamente* raggiungere più persone di quante ne esistano sulla Terra!

Un momento...

Questa è una semplificazione! In realtà i tuoi amici si conoscono tra loro (sovrapposizione), quindi la portata reale è minore. Ecco perché dobbiamo simulare una vera *rete* per vedere cosa succede davvero.

Passo 2: Costruire una Rete Sociale

Costruiremo una **rete small-world** — un modello inventato da Watts e Strogatz nel 1998 che cattura il funzionamento delle reti sociali reali.

```python
import random

def build_network(n_people, n_contacts, rewire_prob=0.1):
    """Build a small-world network."""
    # Partenza: collega ogni persona ai vicini più prossimi
    # (come persone sedute in cerchio che conoscono chi sta accanto)
    network = {i: set() for i in range(n_people)}

    # Collega ai vicini più prossimi (reticolo ad anello)
    for i in range(n_people):
        for j in range(1, n_contacts // 2 + 1):
            neighbor = (i + j) % n_people
            network[i].add(neighbor)
            network[neighbor].add(i)

    # Ricollega casualmente alcune connessioni (crea "scorciatoie")
    for i in range(n_people):
        for j in range(1, n_contacts // 2 + 1):
            if random.random() < rewire_prob:
                neighbor = (i + j) % n_people
                # Rimuovi la vecchia connessione e aggiungine una casuale
                if neighbor in network[i] and len(network[i]) > 1:
                    network[i].discard(neighbor)
                    network[neighbor].discard(i)
                    # Scegli una nuova connessione casuale
                    new = random.randint(0, n_people - 1)
                    while new == i or new in network[i]:
                        new = random.randint(0, n_people - 1)
                    network[i].add(new)
                    network[new].add(i)

    return network
```

Passo 3: Trovare il Cammino Più Breve (BFS)

Per trovare il cammino più breve tra due persone, usiamo la **Breadth-First Search** (BFS) — uno degli algoritmi più importanti dell'informatica:

```python
def shortest_path(network, start, end):
    """Find shortest path using BFS."""
    if start == end:
        return [start]

    visited = {start}
    queue = [(start, [start])]  # (nodo corrente, cammino finora)

    while queue:
        current, path = queue.pop(0)

        for neighbor in network[current]:
            if neighbor == end:
                return path + [neighbor]  # trovato!

            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

```

```
    return []  # nessun cammino trovato

# Costruisci e testa
net = build_network(200, 6, 0.1)
path = shortest_path(net, 0, 100)
print(f"Shortest path from person 0 to person 100:")
print(f"Path: {path}")
print(f"Steps: {len(path) - 1}")
```

Output

```
Shortest path from person 0 to person 100:
Path:  [0, 3, 47, 98, 100]
Steps:  4
```

Passo 4: Misurare Tutte le Distanze

Misuriamo la distanza media tra molte coppie casuali:

```
import random

# Costruisci una rete
N = 500
net = build_network(N, 10, 0.1)

# Campiona coppie casuali e misura le distanze
distances = []
for trial in range(2000):
    a = random.randint(0, N - 1)
    b = random.randint(0, N - 1)
    if a != b:
        path = shortest_path(net, a, b)
        if path:
            distances.append(len(path) - 1)

avg = sum(distances) / len(distances)
print(f"Network: {N} people, 10 contacts each")
print(f"Average distance: {avg:.2f} steps")
print(f"Maximum distance: {max(distances)} steps")
```

Passo 5: Istogramma delle Distanze

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4.5))
plt.hist(distances, bins=range(1, max(distances) + 2),
         color='#16A34A', alpha=0.8, edgecolor='white',
         linewidth=1.2, density=True)
plt.axvline(x=avg, color='#DC2626', linestyle='--', linewidth=2,
            label=f'Mean = {avg:.1f} steps')
plt.xlabel('Shortest path length (steps)', fontsize=12)
plt.ylabel('Fraction of pairs', fontsize=12)
plt.title(f'Six Degrees: Network of {N} People', fontsize=14,
          fontweight='bold')
plt.legend(fontsize=11)
plt.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

L'Effetto Small-World

Anche se ogni persona conosce solo 10 altre persone, il cammino medio è molto breve! Questo accade perché le "scorciatoie" casuali (connessioni ricollegate) riducono drasticamente le distanze. È esattamente ciò che succede nella vita reale: il tuo amico che si è trasferito all'estero ti collega a un circolo sociale completamente diverso.

Sfida: Esplora i Parametri

Prova a cambiare i parametri della rete e osserva:

1. Cosa succede se `rewire_prob = 0`? (Nessuna scorciatoia — un puro anello)
2. Cosa succede se `rewire_prob = 1`? (Rete completamente casuale)
3. Come influisce l'aumento di `n_contacts` sulla distanza media?
4. Riesci a trovare una combinazione in cui la distanza media è inferiore a 3?

Crea un grafico che mostri la distanza media in funzione della probabilità di ricollegamento!

Riepilogo Finale

Hai appena programmato cinque paradossi della probabilità! Creiamo un grafico riassuntivo finale:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

# (a) Monty Hall
ax = axes[0, 0]
# (riesegui la simulazione di Monty Hall dalla Sezione 2)
stay_r, switch_r = [], []
sw, stw = 0, 0
for g in range(1, 5001):
    car = random.randint(0, 2)
    pick = random.randint(0, 2)
    if pick == car: stw += 1
    else: sw += 1
    stay_r.append(stw / g)
    switch_r.append(sw / g)
ax.plot(stay_r, color='#DC2626', linewidth=1)
ax.plot(switch_r, color='#16A34A', linewidth=1)
ax.axhline(1/3, color='#DC2626', linestyle='--', alpha=0.5)
ax.axhline(2/3, color='#16A34A', linestyle='--', alpha=0.5)
ax.set_title('(a) Monty Hall', fontweight='bold')
ax.set_ylabel('Win rate')
ax.set_ylim(0.15, 0.85)
ax.grid(True, alpha=0.3)

# (b) Boy or Girl
ax = axes[0, 1]
boy_cnt, both_cnt = 0, 0
probs = []
for i in range(5000):
    c1 = random.choice([0, 1])
    c2 = random.choice([0, 1])
    if c1 == 1 or c2 == 1:
        boy_cnt += 1
        if c1 == 1 and c2 == 1:
            both_cnt += 1
    probs.append(both_cnt / boy_cnt if boy_cnt > 0 else 0)
ax.plot(probs, color='#2563EB', linewidth=1)
ax.axhline(1/3, color='#DC2626', linestyle='--', linewidth=2)
ax.set_title('(b) Boy or Girl', fontweight='bold')
ax.set_ylim(0.2, 0.5)
ax.grid(True, alpha=0.3)

# (c) Birthday Problem
ax = axes[1, 0]
sizes = list(range(2, 61))
theory = [birthday_theory(n) for n in sizes]
ax.plot(sizes, theory, color='#DC2626', linewidth=2)
ax.axhline(0.5, color='gray', linestyle=':')
ax.axvline(23, color='#16A34A', linestyle='--')
ax.set_title('(c) Birthday Problem', fontweight='bold')
ax.set_xlabel('People in room')
```

```
ax.set_ylabel('P(shared birthday)')
ax.grid(True, alpha=0.3)

# (d) Six Degrees
ax = axes[1, 1]
ax.hist(distances, bins=range(1, max(distances) + 2),
        color='#16A34A', alpha=0.8, edgecolor='white', density=True)
ax.axvline(avg, color='#DC2626', linestyle='--', linewidth=2)
ax.set_title('(d) Six Degrees', fontweight='bold')
ax.set_xlabel('Path length')
ax.grid(axis='y', alpha=0.3)

plt.suptitle('Five Probability Paradoxes -- Simulated!',
             fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```

Cosa Hai Imparato

Abilità Python	Dove l'hai usata
Variabili e matematica	Ovunque!
Cicli `for`	Eseguire migliaia di simulazioni
Condizioni `if`/`else`	Verificare se il giocatore ha vinto
Funzioni	`has_birthday_match()`, `build_network()`
Liste	Memorizzare risultati, compleanni, cammini
Dizionari	Adiacenza della rete, dati raggruppati
Libreria `random`	Simulare eventi casuali
Grafici con `matplotlib`	Grafici di convergenza, barre, istogrammi
Algoritmo BFS	Trovare cammini minimi nelle reti
Simulazione Monte Carlo	Stimare probabilità empiricamente

Punti Chiave

1. **Monty Hall:** Cambia sempre — vinci 2/3 delle volte, non 1/2.
2. **Ragazzo o Ragazza:** *Come* ottieni l'informazione cambia la probabilità. La risposta è 1/3, non 1/2.
3. **Problema del Compleanno:** Con sole 23 persone, P(coincidenza) $> 50\%$. La nostra intuizione sulle combinazioni è pessima.
4. **Paradosso di Simpson:** I dati aggregati possono invertire le tendenze. Cerca sempre le variabili nascoste.
5. **Sei Gradi:** Poche connessioni casuali creano scorciatoie che collegano intere reti. Il mondo è piccolo.

Per Approfondire

- **Più Python:** https://www.codecademy.com/learn/learn-python-3
- **Probabilità:** 3Blue1Brown su YouTube — splendide spiegazioni visive
- **Data Science:** Kaggle.com — esercitati con dati reali
- **Scienza delle Reti:** "Linked" di Albert-László Barabási (libro divulgativo)

Buona programmazione!

Il modo migliore per imparare è *sperimentare*.
Cambia i numeri. Rompi le cose. Chiedi "e se?"
È questo il bello della programmazione — e della scienza.